ПРАКТИЧЕСКИЙ КУРС · ВЕРСИЯ ДЛЯ НАЧИНАЮЩИХ

UIscript

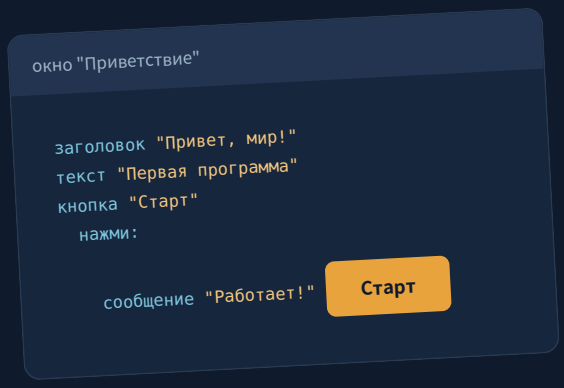
Расширенный самоучитель

Практическое руководство по языку интерактивных интерфейсов: от первой строки до полноценных мини-приложений.

19
глав

4
законченных проекта

Упражнения
с решениями



Основано на среде разработки UIscript версии 1.0 (uis_1-0.html) и исходном самоучителе PDF
Редакция: октябрь 2026 · GromAI Studio

Как пользоваться книгой

Лучший способ изучить UIscript — не читать книгу подряд без практики, а чередовать короткое объяснение, запуск примера и самостоятельное изменение кода. Откройте HTML-файл среды разработки в браузере. В редакторе замените текст примером из книги и нажмите «Запустить» или Ctrl+Enter (на macOS обычно Cmd+Enter). Сначала добейтесь точно такого результата, как в описании; затем поменяйте надписи, числа, цвета и имена.

В книге используется русскоязычный синтаксис, потому что он соответствует примеру по умолчанию в приложенной среде. Английские псевдонимы тоже поддерживаются, но они не превращают UIscript в полноценный JavaScript: набор команд и выражений остаётся своим.

Учебный маршрут

Этап	Что освоить	Проверка результата
1. Основы	Строки, отступы, окно, текст, кнопка	Сделать приветствие с кнопкой
2. Данные	Имена элементов, переменные, выражения	Сложить два введённых числа
3. Поведение	Условия, события, действия	Проверить форму и показать ошибку
4. Повторение	Циклы и функции	Создать таблицу умножения
5. Динамика	Таймеры и реактивность	Сделать счётчик или часы
6. Приложения	Списки, выбор, запросы, экспорт	Собрать законченное мини-приложение

> **Правило практики**
>
> После каждого раздела измените хотя бы одну строку, чтобы проверить, действительно ли вы поняли причину результата.

Содержание

1 Что такое UIscript и как устроена среда

2 Синтаксис: строки, кавычки, комментарии и отступы

3 Элементы интерфейса и вложенность

4 Свойства и оформление

5 Переменные, значения и выражения

6 Условия и проверка ввода

7 События и действия

8 Циклы

9 Функции и область видимости

10 Таймеры

11 Реактивность и условные свойства

12 Поля, выбор, галочки и списки

13 Сеть и API

14 Английские псевдонимы и экспорт HTML

15 Поиск ошибок и отладка

16 Практикум: четыре законченных проекта

17 Проверка качества перед публикацией

18 Полный краткий справочник

19 Задания для самостоятельной работы и решения

01 Что такое UIscript и как устроена среда

UIscript — небольшой предметно-ориентированный язык (DSL) для создания интерактивных окон и мини-приложений в браузере. Вы описываете элементы и их поведение короткими строками, а среда строит интерфейс. Программировать непосредственно на HTML, CSS или JavaScript для базовых задач не требуется.

При этом UIscript — отдельный язык со своим синтаксисом. Например, кнопка выглядит как `кнопка "Старт"`, а обработчик нажатия начинается строкой `нажми:`. Нельзя автоматически применять к UIscript все правила JavaScript: здесь есть `пусть`, `если`, `повторить`, `функция`, но нет произвольного доступа ко всем возможностям браузера из пользовательского кода.

1.1. Части среды разработки

Редактор — место, где пишется программа. Слева показываются номера строк; ниже редактора доступна строка состояния с позицией курсора и количеством символов.

Консоль — журнал сообщений, предупреждений и ошибок. Команда `показать` пишет сюда, а `сообщение` выводит запись в консоль и всплывающее уведомление.

Предпросмотр — результат выполнения программы. Переключатель устройства позволяет посмотреть интерфейс в режиме рабочего стола или мобильного экрана.

Примеры — готовые образцы для изучения языка и шаблоны страниц. Загрузка примера заменяет содержимое редактора; отменить действие можно Ctrl+Z.

Справка — краткая памятка по синтаксису. Она полезна в работе, а этот самоучитель раскрывает темы подробнее.

1.2. Запуск, сохранение и клавиши

Действие	Как выполнить
Запустить код	Кнопка «Запустить» или Ctrl+Enter (Cmd+Enter на Mac)
Сохранить исходник	Ctrl+S или меню «Файл» → «Сохранить код (.uis)»
Загрузить исходник	«Файл» → «Импорт файла…»; поддерживаются .uis и .txt
Скачать приложение	«Файл» → «Скачать приложение (.html)»
Комментировать строку	Ctrl+/
Добавить отступ	Tab
Убрать отступ	Shift+Tab
Закрыть окно или меню	Esc
Отменить действие	Ctrl+Z

Среда автоматически сохраняет текст редактора в хранилище браузера после небольшой паузы в наборе. Это удобно, но не заменяет резервную копию: при смене браузера, очистке данных сайта или проблеме с локальным хранилищем код можно потерять. Важные проекты сохраняйте в отдельный файл .uis.

Пункт «Вид» → «Автозапуск при вводе» включает повторный запуск программы при редактировании. Для крупных примеров или программ с таймерами на первых порах лучше запускать вручную: так проще понимать, когда именно код исполняется заново.

1.3. Первая программа

```
окно "Привет"
    заголовок "Привет, мир!"
    текст "Это моя первая программа на UIscript."
```

1. Вставьте пример в редактор.
2. Нажмите «Запустить».
3. Найдите окно с заголовком «Привет».
4. Поменяйте текст, снова запустите и сравните результат.

Первая строка создаёт окно. Две строки с большим отступом — элементы внутри окна. Так вы уже использовали главное правило языка: отступ показывает структуру.

1.4. Первая интерактивность

```
окно "Знакомство"
    заголовок "Нажми кнопку"
    кнопка "Поздороваться"
        нажми:
            сообщение "Привет! Кнопка сработала."
            показать "Это запись в консоли."
```

Команды внутри `нажми:` выполняются при нажатии кнопки. `сообщение` показывает уведомление; `показать` только добавляет сообщение в консоль. Если событие ничего не делает, проверьте отступы и убедитесь, что действия находятся внутри блока события.

> **Мини-практика**
>
> Добавьте вторую кнопку «Секрет», которая выводит другое сообщение. Не копируйте весь пример заново: скопируйте только блок `кнопка` и поменяйте текст и сообщение.

02 Синтаксис: строки, кавычки, комментарии и отступы

2.1. Одна строка — одна инструкция

Обычно строка содержит один элемент интерфейса, одно свойство, одно событие, одно действие, объявление переменной или управляющую конструкцию. Читайте код сверху вниз: порядок имеет значение. Например, переменную, которую используют для вычисления, следует задать до этого вычисления.

```
пусть имя = "Аня"
пусть приветствие = "Привет, " + имя
окно "Программа"
    текст приветствие
```

Здесь `пусть` создаёт переменные. В окне показывается значение переменной `приветствие`.

2.2. Отступы задают вложенность

Вложенные строки начинаются правее строки, к которой относятся. Обычно принято использовать два пробела на уровень. Важно не конкретное число пробелов, а последовательность и единообразие.

```
окно "Анкета"
    заголовок "Ваши данные"
    строка
        поле имя "Имя"
        поле город "Город"
    кнопка "Готово"
        нажми:
            сообщение "Привет, {имя}!"
```

Строка `поле` вложена в `строка`, а сама `строка` — в `окно`. Действие `сообщение` вложено в событие `нажми`, а событие — в кнопку.

Сравните с ошибочным вариантом:

```
пусть число = 7
    число = 8
```

Присваивание во второй строке не является дочерним элементом переменной, и среда сообщит о неожиданном отступе. Команды вроде `пусть`, присваивание и `показать` не образуют контейнеры. Дополнительный отступ допустим только после конструкции, которая действительно содержит вложенный блок.

2.3. Кавычки и текст

Строки текста обычно записывают в двойных кавычках: `"Привет"`. В выражениях поддерживаются и одинарные кавычки, однако в примерах книги используется единый стиль с двойными кавычками. Внутри строк доступны простые экранированные последовательности:

```
окно "Строки"
    текст "Первая строка\nВторая строка"
    текст "Колонка A\tКолонка B"
```

`\n` означает перевод строки, `\t` — табуляцию. Обратная косая черта позволяет экранировать следующий символ, например кавычку. Если забыть закрывающую кавычку, парсер сообщит о незакрытой строке.

2.4. Комментарии

Комментарий начинается с `//`. Он нужен для пояснений и не выполняется.

```
// Счётчик хранит число нажатий.
пусть счётчик = 0
окно "Счётчик"
    текст "Нажатий: {счётчик}"   // текст обновляется автоматически
    кнопка "+1"
        нажми:
            счётчик += 1
```

Внутри кавычек `//` является обычным текстом. Например, адрес `"https://example.com"` не превращается в комментарий.

2.5. Имена элементов и переменных

Имена — это идентификаторы, по которым программа обращается к элементам и переменным. Допустимы буквы, цифры и подчёркивание; имя не должно начинаться с цифры. Кириллица разрешена. `имя`, `Имя` и `ИМЯ` — разные имена, поэтому придерживайтесь одного регистра.

Хорошие имена: `имя`, `итог`, `число_попыток`, `список_покупок`.
Плохие имена: `2числа`, `цена товара` (пробел внутри имени), `если` (ключевое слово).

Не используйте для переменных и функций слова `если`, `иначе`, `пусть`, `повторить`, `функция`, `вернуть`, а также логические константы `да`, `нет`, `истина`, `ложь`, `true`, `false` и `null`.

Полезная привычка: имя — существительное, действие — глагол, комментарий — поясняет причину. Например, `поле имя`, `кнопка "Рассчитать"`, `// Не считаем пустое значение числом`.

> **Мини-практика**
>
> Напишите программу с двумя строками текста и одной кнопкой. Намеренно уберите отступ у одной дочерней строки, запустите код, найдите ошибку и исправьте её.

03 Элементы интерфейса и вложенность

Элемент — видимая часть приложения: заголовок, поле, список, кнопка или контейнер. Элементы создаются отдельными строками и обычно могут принимать вложенные свойства, события либо дочерние элементы.

3.1. Справочник элементов

Команда	Назначение	Пример
`окно`	Основная панель с заголовком	`окно "Профиль"`
`заголовок`	Крупная надпись	`заголовок "Добро пожаловать"`
`подзаголовок`	Надпись второго уровня	`подзаголовок "Настройки"`
`текст`	Абзац или короткая подпись	`текст "Выберите действие"`
`кнопка`	Кнопка действия	`кнопка "Сохранить"`
`поле`	Однострочный текстовый ввод	`поле имя "Введите имя"`
`область`	Многострочный текстовый ввод	`область заметка "Ваш комментарий"`
`галочка`	Логический переключатель	`галочка согласен "Я согласен"`
`ползунок`	Выбор числа в диапазоне	`ползунок громкость`
`выбор`	Выпадающий список	`выбор город "Выберите город"`
`вариант`	Вариант внутри `выбор`	`вариант "Прага"`
`список`	Список строк, которым можно управлять	`список задачи`
`картинка`	Изображение по URL	`картинка "https://example.com/a.jpg"`
`видео`	Видеоплеер	`видео "https://example.com/video.mp4"`
`ссылка`	Ссылка на ресурс	`ссылка "Документация"`
`разделитель`	Горизонтальная линия	`разделитель`
`строка`	Контейнер: элементы слева направо	`строка`
`столбец`	Контейнер: элементы сверху вниз	`столбец`
`карточка`	Карточка для группировки содержимого	`карточка блок "Итоги"`

> **Важно.** Для изображения или видео нужен доступный URL ресурса. Если изображение не загружается, среда показывает заглушку. Загрузка контента по сети зависит от доступности URL и политик сайта. Видео-плеер отображается с собственными элементами управления браузера.

3.2. Имена элементов: когда они нужны

Имя нужно, если вы хотите читать значение поля, менять текст, включать/выключать элемент, очищать его или менять его видимость. В именованных текстовых элементах имя обычно ставится после

ключевого слова и перед текстом:

```
окно "Приветствие"
    поле имя "Как тебя зовут?"
    текст результат "Здесь появится приветствие"
    кнопка "Показать"
        нажми:
            установить результат "Привет, {имя}!"
```

Здесь `имя` — имя поля, `результат` — имя элемента `текст`. Подпись поля — это подсказка внутри поля, а не отдельный постоянный текст.

Ввод без имени тоже допустим, если значение не понадобится читать в программе. Но если вы собираетесь использовать поле в условии или вычислениях, сразу дайте ему понятное имя.

3.3. Контейнеры и визуальная структура

`окно`, `строка`, `столбец` и `карточка` помогают группировать элементы. В строке дочерние элементы располагаются рядом, в столбце — друг под другом. Карточка визуально обособляет содержимое.

```
окно "Профиль"
    заголовок "Карточка пользователя"
    строка
        столбец
            текст имя_подпись "Имя"
            поле имя "Алексей"
            текст город_подпись "Город"
            поле город "Бухарест"
        карточка информация "Сводка"
            текст "Данные можно изменить слева."
    кнопка "Показать данные"
        нажми:
            сообщение "{имя}, город: {город}"
```

Не стоит помещать в один ряд слишком много широких полей: на небольшом экране им может не хватить места. Для формы с несколькими параметрами удобнее столбец или несколько вложенных строк.

3.4. Ползунок

У ползунка можно указать нижнюю и верхнюю границы, а также начальное значение. Имя элемента даёт доступ к его текущему значению.

```
окно "Громкость"
    ползунок громкость
        мин 0
        макс 100
        значение 35
    текст "Громкость: {громкость}%"
```

Если значение используется в арифметике, проверьте, что оно находится в ожидаемом диапазоне. Сам ползунок ограничивает выбор заданными границами.

3.5. Выпадающий список

Варианты должны находиться с отступом внутри `выбор`.

```
окно "Город"
    выбор город "Выберите город"
        вариант "Бухарест"
        вариант "Клуж"
        вариант "Яссы"
    кнопка "Выбрать"
        нажми:
            если город == ""
                сообщение "Сначала выберите город."
            иначе
                сообщение "Вы выбрали город: {город}"
```

Если в `выбор` указана подсказка, начальным значением будет пустая строка. Это позволяет проверить, сделал ли пользователь выбор.

3.6. Изображения, ссылки и адреса

Изображение можно вставить напрямую:

```
окно "Фото"
    картинка "https://picsum.photos/640/360"
        ширина 640
        скругление 12
```

Ссылка и медиа-элемент могут использовать свойство `адрес`. Для веб-ссылок среда разрешает HTTP(S), `mailto:`, `tel:` и некоторые относительные пути. Не указывайте неизвестные схемы, а для внешних ссылок выбирайте адрес, которому доверяете.

```
окно "Полезное"
    ссылка "Открыть сайт"
        адрес "https://example.com"
```

> **Мини-практика**
>
> Сделайте карточку профиля с заголовком, двумя полями, выпадающим списком города и кнопкой. Пока кнопка может лишь показывать сообщение с введёнными значениями.

04 Свойства и оформление

Свойства настраивают внешний вид уже созданного элемента. Свойство пишется с отступом под элементом, к которому относится. Если свойство стоит на верхнем уровне, среда не сможет определить его владельца и покажет предупреждение.

4.1. Полный список поддерживаемых свойств

Свойство	Значение	Что меняет / где полезно
`фон`	Цвет	Цвет фона элемента
`цвет`	Цвет	Цвет текста
`размер`	Число	Размер текста в пикселях
`жирный`	Флаг	Жирное начертание
`курсив`	Флаг	Курсивное начертание
`центр`	Флаг	Выравнивание текста по центру
`подсказка`	Текст	Placeholder у поля/области или всплывающая подсказка у других элементов
`ширина`	Число или CSS-размер	Ширина элемента
`высота`	Число или CSS-размер	Высота элемента
`отступ`	Число или CSS-размер	Внутренний отступ; у строки/столбца также расстояние между дочерними элементами
`скругление`	Число или CSS-размер	Радиус углов
`рамка`	Цвет или описание границы	Рамка вокруг элемента
`тень`	Флаг	Тень элемента
`прозрачность`	Число	Прозрачность, обычно от 0 до 1
`невидимый`	Флаг	Скрывает элемент
`мин`	Число	Нижняя граница ползунка или совместимого элемента управления
`макс`	Число	Верхняя граница ползунка или совместимого элемента управления
`значение`	Текст/число	Начальное значение поля/контрола или текста у поддерживаемых надписей
`адрес`	URL	Адрес ссылки, картинки или видео
`выключен`	Флаг	Блокирует элемент управления, если он поддерживает состояние disabled

Число в `ширина`, `высота`, `отступ` и `скругление` трактуется как пиксели. Примеры: `ширина 240`, `высота 80`, `скругление 12`. Для цвета используйте формат CSS: `"#1d4ed8"`, `"#fef3c7"`, `"red"`.

4.2. Значения-флаги

Флаг без значения включает свойство:

```
окно "Акцент"
    карточка итог "Итого"
        фон "#eff6ff"
        рамка "#93c5fd"
        скругление 14
        отступ 16
        тень
        текст "Сумма заказа"
            жирный
        текст "125 леев"
            размер 28
            цвет "#1d4ed8"
```

Флаги могут также получать явно заданное логическое значение, когда это поддерживается. `жирный` без значения означает включённое состояние, `жирный ложь` выключит его. Для читаемости чаще используйте обычный флаг либо условную форму из главы 11.

4.3. Рамка и размеры

```
окно "Оформление"
    карточка примечание "Важно"
        фон "#fff7ed"
        рамка "#fb923c"
        скругление 16
        отступ 18
        текст "Проверьте введённые значения перед расчётом."
            цвет "#9a3412"
        разделитель
    кнопка "Продолжить"
        ширина 220
        высота 44
```

Для рамки чаще всего достаточно указать цвет: `рамка "#3b82f6"`. Можно задать полное строковое описание границы, например `рамка "2px solid #3b82f6"`. Тень включается как флаг `тень`.

4.4. Не все свойства подходят любому элементу

Некоторые свойства имеют смысл только на определённых элементах. `мин`, `макс` и `значение` предназначены для контролов; `адрес` — для ссылки, картинки и видео; `выключен` — для элементов, которые в браузере могут блокироваться. Если применить их к неподходящему элементу, команда не обязательно остановит программу, но в консоли будет предупреждение.

> **Мини-практика**
>
> Оформите карточку в собственном стиле: фон, рамка, скругление, внутренний отступ, заголовок и кнопка. Затем намеренно примените `выключен` к обычному тексту и прочитайте предупреждение в консоли.

05 Переменные, значения и выражения

Переменная — именованное место, где программа хранит значение. В отличие от текста, который всегда одинаков, значение переменной можно пересчитывать в ходе работы.

5.1. Создание, присваивание и изменение

```
пусть очки = 10
очки = очки + 5
очки += 2
окно "Очки"
    текст "Итого: {очки}"
```

`пусть очки = 10` создаёт переменную. `очки = очки + 5` вычисляет правую часть и записывает результат обратно. Короткие формы `+=`, `-=`, `*=`, `/=` и `%=` упрощают изменение на месте.

Переменную можно объявить без начального значения: `пусть заметка`. Тогда она имеет значение `null`. Команда `очистить заметка` тоже делает переменную пустой (`null`).

Различайте присваивание и сравнение: одиночный `=` присваивает значение, а `==` сравнивает два значения. В условии допустим `=`, но среда предупреждает, что обычно следует писать `==`.

5.2. Виды значений

Вид	Примеры	Примечание
Число	`12`, `3.14`, `-5`	Дробная часть отделяется точкой в коде
Строка	`"Привет"`	Текст в кавычках
Логическое значение	`истина`, `ложь`	Также принимаются `true` и `false`, `да` и `нет`
Пустое значение	`null`	Значение не задано

В поле пользователь вводит текст. Даже если он набрал `12`, значение обычного текстового поля — строка, а не число. Перед арифметикой используйте `число(имя_поля)`.

5.3. Арифметические операторы

Оператор	Назначение	Пример → результат
`+`	Сложение или склеивание строк	`2 + 3` → 5
`-`	Вычитание	`9 - 4` → 5
`*`	Умножение	`6 * 7` → 42
`/`	Деление	`7 / 2` → 3.5
`%`	Остаток от деления	`10 % 3` → 1

Если одним из операндов `+` является строка, значения соединяются как текст. Из-за этого выражение

`"Сумма: " + 2 + 3` даёт текст `Сумма: 23`. Для числового результата сначала выполните арифметику: `"Сумма: " + (2 + 3)`.

5.4. Сравнения и логика

Операторы	Что делают
`==`, `=`	Проверяют равенство
`!=`	Проверяет неравенство
`>`, `<`, `>=`, `<=`	Сравнивают порядок
`и` / `and` / `&&`	Истина, если истинны обе части
`или` / `or` / `||`	Истина, если истинна хотя бы одна часть
`не` / `not` / `!`	Отрицание логического значения

Сначала считаются скобки, затем умножение, деление и остаток, затем сложение и вычитание, сравнения и логические операции. Когда выражение становится трудно читать, добавляйте круглые скобки.

UIscript сравнивает числовые строки по числовому значению, когда обе стороны выглядят как числа: `"10" > "9"` истинно. При этом сравнение строк на равенство сохраняет строковый смысл: `"007" == "7"` ложно. Пустая строка не должна считаться нулём; проверяйте пустой ввод отдельно.

5.5. Преобразование ввода в число

Неправильно:

```
окно "Сумма"
    поле а "Первое число"
    поле б "Второе число"
    кнопка "Сложить"
        нажми:
            пусть сумма = а + б
            сообщение "Сумма: {сумма}"
```

Если в полях введены 2 и 3, результатом будет `23`: программа склеит строки.

Правильно:

```
окно "Сумма"
    поле а "Первое число"
    поле б "Второе число"
    кнопка "Сложить"
        нажми:
            пусть сумма = число(а) + число(б)
            сообщение "Сумма: {сумма}"
```

Функция `число()` умеет читать десятичную запятую в пользовательском вводе: `число("1,5")` даёт `1.5`. Если текст нельзя преобразовать в число, результатом будет `NaN` — «не число». Поэтому сначала проверяйте данные функцией `число_ли()`.

5.6. Подстановка в текст

Фигурные скобки внутри строки подставляют значение переменной или результат выражения:

```
пусть имя = "Лена"
пусть цена = 12.5
окно "Чек"
    текст "Покупатель: {имя}"
    текст "Цена с округлением: {округлить(цена, 1)}"
```

Внутри скобок можно использовать арифметику и встроенные функции. Если имя не определено, подстановка обычно превращается в пустой фрагмент, поэтому проверяйте написание идентификаторов.

> **Мини-практика**
>
> Создайте конвертер градусов: поле температуры, кнопка и результат по формуле `(число(температура) * 9 / 5) + 32`. Добавьте `округлить(…, 1)`, чтобы не выводить слишком много дробных знаков.

06 Условия и проверка ввода

Условие позволяет выбрать действие в зависимости от данных. Оно особенно полезно для форм и калькуляторов: прежде чем считать, нужно убедиться, что пользователь ввёл допустимые значения.

6.1. если, иначе если, иначе

```
пусть оценка = 4
если оценка == 5
    сообщение "Отлично!"
иначе если оценка == 4
    сообщение "Хорошо."
иначе если оценка == 3
    сообщение "Удовлетворительно."
иначе
    сообщение "Нужно повторить материал."
```

Проверки идут сверху вниз. Выполняется только первая ветка с истинным условием. Блок `иначе` необязателен, но слово `иначе` должно соответствовать `если` по отступу. В этой среде после `если` двоеточие не требуется, но допускается; в примерах книги часто используется именно этот стиль.

6.2. Составные условия

```
если число_ли(возраст) и число(возраст) >= 18
    сообщение "Возраст подходит."
иначе
    сообщение "Введите возраст от 18 лет."
```

`и` требует, чтобы обе части были истинны. `или` достаточно одной истинной части. `не` меняет логическое значение на противоположное. Среда поддерживает также `and`, `or`, `not`, `&&`, `||` и `!`.

Выражения с `и` и `или` вычисляются с коротким замыканием: если результат уже известен по первой части, вторая часть может не вычисляться. Это позволяет писать `пусто(имя) или длина(имя) < 2`: при пустом имени проверка длины не нужна.

6.3. Истинность значения

В условии можно использовать не только сравнение, но и само значение. Обычно истинными считаются непустые значения, отличные от `0`, `ложь` и `null`. У галочки значение логическое, поэтому естественно написать `если согласен`.

> **Осторожно.** Строка `"0"` — это непустой текст, и как строковое значение она считается истинной. Если проверяете число, преобразуйте его: `если число(ввод) != 0`.

6.4. Четыре полезные функции валидации

Функция	Что делает	Пример
`число_ли(x)`	Проверяет, читается ли значение как конечное число	`число_ли("12,5")` → истина
`пусто(x)`	Проверяет отсутствие значения; учитывает пустую строку, `null`, `ложь` и пустой массив	`пусто("")` → истина
`обрезать(x)`	Удаляет пробелы по краям текстовой строки	`обрезать(" да ")` → "да"
`длина(x)`	Возвращает длину строки или массива	`длина("привет")` → 6

6.5. Проверка имени

```
окно "Регистрация"
    поле имя "Ваше имя"
    галочка согласен "Принимаю условия"
    кнопка "Продолжить"
        нажми:
            если обрезать(имя) == ""
                сообщение "Введите имя."
                фокус имя
            иначе если не согласен
                сообщение "Подтвердите согласие."
            иначе
                сообщение "Добро пожаловать, {обрезать(имя)}!"
```

`обрезать(имя) == ""` ловит не только пустое поле, но и поле, в котором пользователь ввёл одни пробелы. `фокус имя` возвращает курсор в нужное поле.

6.6. Безопасное числовое вычисление

```
окно "Квадрат числа"
    поле ввод "Введите число, например 7"
    текст результат "Результат появится здесь"
    кнопка "Посчитать"
        нажми:
            если не число_ли(ввод)
                сообщение "Введите корректное число."
                фокус ввод
            иначе
                пусть x = число(ввод)
                установить результат "Квадрат: {округлить(x * x, 2)}"
```

Главная последовательность такая: сначала проверка, затем преобразование, после этого вычисление и вывод. Не умножайте результат `число(ввод)` на себя, пока не проверили `число_ли(ввод)`: иначе `NaN` может попасть в последующие вычисления.

> **Мини-практика**
>
> Расширьте проверку имени: не принимать имя короче двух символов. Затем добавьте проверку, что сумма в поле `сумма` — положительное число.

07 События и действия

Событие — момент, когда пользователь взаимодействует с элементом. Действия внутри блока события выполняются в ответ на взаимодействие.

7.1. События

Событие	Когда срабатывает	Пример использования
`нажми:`	Щелчок или нажатие на элемент	Кнопка отправки формы
`измени:`	Изменение значения поля, области, выбора, галочки или ползунка	Пересчёт при вводе
`наведи:`	Курсор вошёл в элемент	Подсказка при наведении мыши
`уход:`	Курсор покинул элемент	Убрать подсветку после наведения

> **Важно.** События `наведи` и `уход` ориентированы на указатель мыши: на устройстве с сенсорным экраном у пользователя может не быть привычного состояния наведения. Для важной функции не делайте hover единственным способом доступа.

7.2. Действия

Команда	Назначение
`сообщение "…"`	Всплывающее уведомление и запись в консоль
`показать "…"`	Запись только в консоль
`установить имя значение`	Задать значение элементу или переменной
`очистить имя`	Очистить поле, список, текст; переменную сделать `null`
`добавить список значение`	Добавить строку в именованный список
`удалить список индекс`	Удалить элемент списка по индексу
`скрыть имя`	Скрыть именованный элемент
`открыть имя`	Показать элемент, ранее скрытый вручную
`переключить имя`	Переключить ручную видимость элемента
`включить имя` / `отключить имя`	Включить или заблокировать подходящий элемент управления
`фокус имя`	Передать фокус элементу, например текстовому полю
`запрос … в переменная`	Отправить HTTP(S)-запрос; подробнее в главе 13
`вызвать функция(…)`	Вызвать собственную функцию, если результат не нужен как выражение

7.3. Почему сообщение и показать разные

```
окно "Журнал"
    кнопка "Проверить"
        нажми:
            показать "Событие зарегистрировано"
            сообщение "Пользователь нажал кнопку"
```

Первую запись ищите в консоли. Вторая тоже появляется в консоли, но дополнительно выводится пользователю как уведомление.

7.4. Установка значения

```
окно "Статус"
    текст статус "Ожидание"
    кнопка "Завершить"
        нажми:
            установить статус "Готово"
            установить прогресс 100
            показать "Статус изменён"
```

Если имя после `установить` совпадает с именем элемента, команда меняет его значение; если с именем переменной — записывает значение переменной. Для читаемости избегайте ситуации, когда переменная и элемент называются одинаково: такое совпадение может затруднить доступ к элементу.

Есть важная особенность: явное `установить` текста реактивного элемента заменяет его вычисляемое содержимое и отключает автоматическое обновление этого текста. Если надпись должна постоянно отражать переменную, используйте строку с фигурными скобками и меняйте переменную, а не текст вручную. См. главу 11.

7.5. Скрытие, включение и фокус

```
окно "Настройки"
    текст расширенные "Дополнительные настройки"
    поле ключ "Введите код"
    кнопка "Показать настройки"
        нажми:
            переключить расширенные
    кнопка "Очистить код"
        нажми:
            очистить ключ
            фокус ключ
```

`скрыть`, `открыть` и `переключить` меняют ручную видимость. Они не удаляют элемент из программы, и у скрытого элемента сохраняется его значение. `отключить` блокирует взаимодействие, если у элемента есть поддержка disabled; `включить` снимает блокировку.

> **Мини-практика**
>
> Создайте переключатель «Показать детали». По нажатию он должен скрывать и открывать карточку, не удаляя её содержимое.

08 Циклы

Цикл повторяет блок кода заданное число раз. В UIscript используется конструкция `повторить N раз:`. Она подходит для таблиц, последовательностей кнопок, списков и небольших повторяющихся расчётов.

8.1. Сумма последовательности

```
пусть сумма = 0
повторить 100 раз:
    сумма += i
окно "Сумма"
    заголовок "Сумма от 1 до 100"
    текст "Результат: {сумма}"
```

Внутри цикла переменная `i` принимает значения от 1 до N. Для примера сумма получится 5050. Цикл с нулём повторов ничего не выполняет. Число повторов может быть выражением, но оно должно вычислиться в число.

8.2. i и индекс

Проход	`i`	`индекс`
Первый	1	0
Второй	2	1
Десятый	10	9

`i` удобен для нумерации человеку, `индекс` — когда нужен отсчёт с нуля. Во вложенных циклах внутренний цикл временно использует свои `i` и `индекс`, поэтому при необходимости сохраните внешний номер в отдельную переменную до запуска внутреннего цикла.

8.3. Таблица умножения

```
окно "Таблица умножения"
    поле множитель "Введите число"
    кнопка "Построить"
        нажми:
            если не число_ли(множитель)
                сообщение "Введите число, например 7."
            иначе
                пусть число_таблицы = число(множитель)
                очистить строки
                повторить 10 раз:
                    добавить строки "{число_таблицы} × {i} = {округлить(число_таблицы * i, 4)}"
    список строки
```

Здесь цикл не только считает числа, но и создаёт содержимое списка. `очистить строки` очищает прежний результат, чтобы таблица не удваивалась после каждого нажатия.

8.4. Генерация ряда элементов

```
окно "Динамические кнопки"
    строка
        повторить 5 раз:
            кнопка "Кнопка {i}"
                нажми:
                    сообщение "Номер кнопки: {i}"
```

Цикл может создавать элементы внутри окна или контейнера. В момент создания элемента среда сохраняет снимок локальных значений (`i` , `индекс` , параметры функции); обработчик потом видит именно этот снимок, а не текущее значение переменной. Это хороший способ генерировать повторяющийся интерфейс, когда не хочется копировать одинаковые строки.

8.5. Ограничения цикла

- На один цикл допускается максимум 10 000 повторений.
- При нечисловом количестве повторов будет ошибка.
- В UIscript нет `while` , `break` и `continue` .

Чтобы завершить повторение досрочно по найденному условию, можно вынести цикл в функцию и выполнить `вернуть` , когда результат найден.

> **Мини-практика**
> Создайте генератор задач: пользователь вводит количество, нажимает кнопку, и программа добавляет в список строки «Задача 1», «Задача 2» и так далее. Проверяйте, что количество — корректное число и не превышает разумный для интерфейса предел.

09 Функции и область видимости

Функция — именованный блок кода, который можно использовать повторно. Она принимает параметры, выполняет действия и при необходимости возвращает результат.

9.1. Объявление и вызов

```
функция квадрат(x):
    вернуть x * x

пусть результат = квадрат(7)
окно "Функция"
    текст "Квадрат 7: {результат}"
```

`функция квадрат(x):` описывает функцию с параметром `x`. `вернуть` завершает работу функции и отдаёт значение тому месту, где функция была вызвана. Если функция ничего не возвращает, результат вызова — undefined (пустое значение).

Если результат не требуется и функцию нужно вызвать как отдельную команду, используйте `вызвать`:

```
функция привет(имя):
    сообщение "Здравствуйте, {имя}!"

окно "Приветствие"
    кнопка "Сказать привет"
        нажми:
            вызвать привет("Мир")
```

9.2. Параметры и локальные переменные

```
пусть x = 1
функция увеличить(x):
    пусть временно = x + 1
    вернуть временно

окно "Локальные значения"
    текст "Результат: {увеличить(10)}"
    текст "Внешняя x: {x}"
```

Параметры и переменные, объявленные внутри функции с помощью `пусть`, локальны для вызова. В этом примере внешняя переменная `x` останется равной 1. Не рассчитывайте, что переменная, впервые созданная внутри функции, останется доступной снаружи.

Функции при этом могут обращаться к глобальным переменным и изменять их, если это необходимо:

```
пусть очки = 0
функция добавить_очки(количество):
    очки += количество

окно "Очки"
    текст "Всего: {очки}"
    кнопка "+5"
        нажми:
            вызвать добавить_очки(5)
```

9.3. Ранний возврат

`вернуть` заканчивает функцию немедленно. Это полезно и в циклах:

```
функция первое_число_больше(n):
    повторить 100 раз:
        если i > n
            вернуть i
    вернуть 0

окно "Поиск"
    текст "Для 7: {первое_число_больше(7)}"
```

Поскольку `вернуть` выходит из функции, цикл прекращается вместе с ней. Добавьте условие выхода в конце функции, чтобы для любого допустимого входа был понятный результат.

9.4. Рекурсия

Функция может вызывать сама себя. Это называется рекурсией. У неё обязательно должно быть условие завершения:

```
функция факториал(n):
    если n <= 1
        вернуть 1
    вернуть n * факториал(n - 1)

окно "Факториал"
    текст "5! = {факториал(5)}"
```

В примере 5! равен 120. В среде установлено ограничение глубины вложенных вызовов — 100. Для задач, которые проще решить циклом, цикл обычно понятнее и безопаснее.

9.5. Частые ошибки в функциях

- Вызвать функцию как выражение можно напрямую: `квадрат(5)`. Но если вызов оформлен отдельной командой, пишите `вызвать квадрат(5)`.
- `вернуть` разрешён только внутри функции.
- Не называйте свою функцию именем встроенной, например `длина` или `число`.
- Следите за скобками и запятыми в списке параметров.
- Не создавайте бесконечную рекурсию: каждый путь должен привести к базовому случаю.

Мини-практика

Напишите функцию `скидка(цена, процент)`, которая возвращает цену после скидки. Вызовите её для нескольких пар чисел и выводите результат с помощью `округлить`.

10 Таймеры

Таймеры выполняют действия по времени. Повторяющийся таймер запускается через заданный интервал, а таймер `через` — один раз после задержки.

10.1. Виды таймеров

Запись	Поведение
`каждую секунду:`	Запускает блок примерно раз в секунду
`каждые 5 сек:`	Повторяет блок каждые 5 секунд
`каждые 500 мс:`	Повторяет блок каждые 500 миллисекунд
`через 3 сек:`	Запускает блок однократно примерно через 3 секунды

В приложенной версии распознаются единицы секунд и миллисекунд: `сек` / варианты со словом «секунд» и `мс` / варианты «миллисекунд», а также английские варианты единиц. Для учебного кода придерживайтесь коротких форм `сек` и `мс`. Интервал должен быть положительным числом. Для повторяющегося таймера минимальный интервал ограничен 50 мс, а одновременно запускается не более 50 таймеров.

10.2. Счётчик секунд

```
пусть тики = 0
окно "Таймер"
    заголовок "Прошло секунд: {тики}"
    каждую секунду:
        тики += 1
```

Каждый тик меняет переменную. Заголовок обновляется благодаря реактивности: таймеру не нужно вручную устанавливать текст заголовка.

10.3. Часы

```
окно "Часы"
    заголовок "{сейчас()}"
        центр
        размер 44
    текст "Сегодня: {дата()}"
        центр
    каждую секунду:
```

Функции `сейчас()` и `дата()` возвращают время и дату по часам устройства. Таймер без действий — допустимый приём: он заставляет реактивные надписи периодически пересчитываться, даже когда переменные программы не меняются.

10.4. Однократная задержка

```
окно "Напоминание"
    кнопка "Напомнить через 3 секунды"
        нажми:
            сообщение "Таймер запущен."
            через 3 сек:
                сообщение "Три секунды прошли!"
```

Тело `через 3 сек:` выполняется один раз. При повторном запуске всей программы таймеры старого запуска прекращаются, а новые создаются заново. Таймеры работают, пока открыта страница, на которой выполняется программа.

> **Осторожно с частыми таймерами.** Не обновляйте интерфейс слишком часто без причины. `каждые 500 мс` подходит для несложного эффекта, но таймер с частым интервалом и дорогими вычислениями может сделать страницу менее отзывчивой. Для учебных счётчиков обычно достаточно `каждую секунду`.

Мини-практика

Сделайте обратный отсчёт с числом «осталось». При нажатии «Старт» уменьшайте его раз в секунду. Когда значение достигнет нуля, покажите сообщение и перестаньте уменьшать счётчик, используя логическую переменную «идёт».

11 Реактивность и условные свойства

Реактивность означает, что отображаемый текст обновляется сам, когда меняются данные, от которых он зависит. В простом случае достаточно поместить имя переменной в фигурные скобки.

11.1. Реактивный текст

```
пусть счётчик = 0
окно "Счётчик"
    заголовок "{счётчик}"
        центр
        размер 48
    строка
        кнопка "−"
            нажми:
                счётчик -= 1
        кнопка "Сброс"
            нажми:
                счётчик = 0
        кнопка "+"
            нажми:
                счётчик += 1
```

Заголовок обновляется после присваивания переменной. Команда `установить заголовок ...` не нужна; более того, ручная установка текста заменит реактивную формулу, и надпись перестанет следить за переменной.

Реактивными могут быть окно или карточка с динамическим заголовком, заголовок, подзаголовок, текст, кнопка, ссылка и подпись галочки. Значения полей также могут использоваться внутри подстановок, поэтому интерфейс способен менять текст при вводе.

11.2. Реактивное вычисление

```
окно "Цена"
    поле цена "Сумма заказа"
    ползунок скидка
        мин 0
        макс 50
        значение 10
    текст "Скидка: {скидка}%"
    текст "К оплате: {округлить(число(цена) * (1 - скидка / 100), 2)}"
```

Если пользователь меняет поле цены или положение ползунка, надпись пересчитывается. Реактивная формула должна быть выражением, которое можно вычислить; используйте проверку `число_ли(цена)`, если пустой ввод не должен давать `NaN`.

11.3. Условные свойства

В приложенной реализации условную форму свойства `если` условие поддерживают пять свойств: `выключен`, `невидимый`, `жирный`, `курсив` и `центр`.

```
пусть попытки = 0
окно "Лимит"
    текст предупреждение "Лимит достигнут"
        цвет "#dc2626"
        жирный если попытки >= 5
        невидимый если попытки < 5
    кнопка "Ещё попытка"
        нажми:
            попытки += 1
```

Нельзя написать `цвет если …` или `фон если …`: для динамического оформления через условие поддерживаются только перечисленные выше флаги. Чтобы выбирать цвет, можно использовать две надписи с разными цветами и управлять их видимостью условием.

11.4. Ручная и условная видимость

У элемента может быть ручное состояние видимости (`скрыть` / `открыть`) и условие `невидимый если …`. Если нужен простой условный показ или скрытие, лучше управлять именно условием. Если пользователь вручную переключает панель, используйте действия `скрыть`, `открыть`, `переключить`. Не смешивайте оба подхода без необходимости: тогда поведение станет сложнее объяснить.

> **Мини-практика**
> Сделайте поле суммы и две надписи: «Введите сумму» — видна при пустом или нечисловом вводе, а итог — виден только при корректном числе. Используйте `невидимый если` и `число_ли`.

12 Поля, выбор, галочки и списки

Элементы ввода позволяют превратить декоративную страницу в приложение. У каждого именованного контрола есть значение, которое можно прочитать по имени.

12.1. Однострочное и многострочное поле

```
окно "Заметка"
    поле тема "Тема заметки"
    область текст_заметки "Напишите текст заметки"
    кнопка "Показать"
        нажми:
            если пусто(обрезать(тема))
                сообщение "Введите тему."
            иначе
                сообщение "Тема: {обрезать(тема)}; заметка: {текст_заметки}"
```

`поле` предназначено для одной строки, `область` — для нескольких. При чтении значения оба возвращают введённый текст. Преобразуйте значение в число только в тех местах, где действительно требуется арифметика.

12.2. Галочка

Галочка возвращает логическое значение: `истина`, если отмечена, и `ложь`, если нет.

```
окно "Подтверждение"
    галочка согласен "Я согласен с условиями"
    кнопка "Продолжить"
        нажми:
            если согласен
                сообщение "Продолжаем."
            иначе
                сообщение "Сначала отметьте галочку."
```

Состояние можно менять через `установить согласен истина` или `установить согласен ложь`. `очистить согласен` снимает отметку. Проверяйте логические флаги без сравнения с `истина`, если выражение не становится менее понятным.

12.3. Выпадающий выбор

Значение `выбор` — текст выбранного варианта. Если выбор не сделан и есть подсказка, значение пустое.

```
окно "Заказ"
    выбор размер "Выберите размер"
        вариант "Маленький"
        вариант "Средний"
        вариант "Большой"
    галочка доставка "Нужна доставка"
    кнопка "Оформить"
        нажми:
            если размер == ""
                сообщение "Выберите размер."
            иначе если доставка
                сообщение "Заказ: {размер}, с доставкой."
            иначе
                сообщение "Заказ: {размер}, самовывоз."
```

12.4. Список строк

`список` отличается от `выбор`: список позволяет добавлять и удалять строки программно, а его значение можно рассматривать как массив текстовых элементов.

```
окно "Список покупок"
    поле товар "Что купить?"
    кнопка "Добавить"
        нажми:
            если пусто(обрезать(товар))
                сообщение "Введите название товара."
            иначе
                добавить покупки "{обрезать(товар)}"
                очистить товар
                фокус товар
    список покупки
    текст "Всего позиций: {длина(покупки)}"
    строка
        кнопка "Удалить первый"
            нажми:
                удалить покупки 0
        кнопка "Удалить последний"
            нажми:
                удалить покупки -1
        кнопка "Очистить всё"
            нажми:
                очистить покупки
```

`добавить покупки …` добавляет новую строку. `удалить покупки 0` удаляет первую строку, а `удалить покупки -1` — последнюю. `очистить покупки` полностью очищает список. Индексация удаления начинается с нуля; отрицательный индекс `-1` используется для последнего элемента.

12.5. Установка и очистка значений

- `установить город "Прага"` выбирает вариант «Прага», если он есть в списке.
- `установить доставка истина` отмечает галочку.
- `установить товар "Молоко"` задаёт текст поля.
- `очистить товар` очищает поле.
- `очистить покупки` очищает список.

`установить` подходит для программного управления значением, а `очистить` — для сброса. Не забывайте, что поле ввода и переменная — разные вещи: поле читается через имя элемента, но в арифметике обычно требует `число(поле)`.

Мини-практика

Создайте список дел с полем, кнопкой добавления, количеством дел, удалением последней строки и полной очисткой. Дополнительное условие: пустые и состоящие только из пробелов строки не должны добавляться.

13 Сеть и API

Команда `запрос` позволяет получить данные с сервера или отправить ему данные. Запрос выполняется асинхронно: программа продолжает работу, а переменная назначения получает результат после завершения запроса.

13.1. Базовый GET-запрос

```
пусть ответ = "Ожидание"
окно "Данные сервера"
    текст "Ответ: {ответ}"
    кнопка "Загрузить"
        нажми:
            ответ = "Загрузка…"
            запрос "https://api.example.com/message" в ответ
```

`api.example.com` в примере — шаблонный адрес, а не готовый источник данных. Замените его URL реального API, который возвращает ответ и разрешает запросы с вашей страницы. Если ответ является JSON, переменная обычно содержит объект; если сервер отдаёт обычный текст, значение будет строкой.

13.2. Доступ к вложенным данным: путь()

Функция `путь(объект, "a.b[0]")` читает вложенное свойство из объекта и может обращаться к элементам массива через индекс в квадратных скобках.

Допустим, сервер вернул JSON вида:

```
{
  "user": { "name": "Mira" },
  "items": ["Книга", "Ручка"]
}
```

Тогда полезны выражения `путь(ответ, "user.name")` и `путь(ответ, "items[0]")`. Поле существует только если оно действительно пришло от сервера; несуществующее поле даёт пустое или неопределённое значение.

13.3. Метод и тело запроса

Действие допускает дополнительные параметры метода и тела, например `запрос "URL" метод "POST" тело "текст" в ответ`. По умолчанию используется GET. Внутренний механизм передаёт тело запроса для методов, отличных от GET и HEAD; объектное тело кодируется как JSON. Синтаксис языка не предоставляет отдельной команды для произвольного набора HTTP-заголовков, поэтому API, которому нужны специальные заголовки или сложная авторизация, может быть неудобен для прямого вызова из UIscript.

13.4. Ограничения сетевых запросов

- Разрешены только URL с протоколами `http://` и `https://`.
- Браузер подчиняется политике CORS. Если сервер не разрешает запрос с вашего источника, среда не может обойти ограничение.

- Тайм-аут запроса в приложенной реализации — около 15 секунд.
- Допускается не более 10 запросов одновременно.

При ошибке HTTP или сети целевая переменная может получить `null`; подробности ищите в консоли. Если сервер требует ключ API, не помещайте секретный ключ в публичную страницу, которую вы будете распространять.

13.5. Как отлаживать запрос

1. Проверьте, что URL открывается и правильный метод поддерживается сервером.
2. Откройте консоль разработчика браузера (F12 на настольных браузерах), если нужны подробности сетевой ошибки.
3. Проверьте тип ответа: строка это или JSON-объект.
4. Для JSON начните с вывода всего ответа, затем используйте `путь()` для нужного поля.
5. Если сообщение указывает на CORS, исправлять нужно конфигурацию сервера или API, а не выражение в UIscript.

> **Мини-практика**
> Найдите публичный API, который явно разрешает браузерные запросы без секретного ключа. Сначала выведите весь ответ, затем извлеките одно поле через `путь()`. Используйте только сервис, которому доверяете.

14 Английские псевдонимы и экспорт HTML

14.1. Английские варианты команд

Среда поддерживает ряд английских псевдонимов для команд, элементов, свойств и событий. Например, `window` соответствует `окно`, `text` — `текст`, `button` — `кнопка`, `click` — `нажми`, `if` — `если`, `let` — `пусть`, `function` — `функция`, `return` — `вернуть`, `show` — `показать`, `message` — `сообщение`, `background` — `фон`.

```
window "Greeting"
    heading "Hello!"
    button "Say hello"
        click:
            message "Hello from UIscript!"
```

Для начала лучше выбрать один стиль и придерживаться его. В русском учебном коде используйте русские команды, а английские псевдонимы — когда читаете готовый пример. Надписи в кавычках не переводятся автоматически.

14.2. Ссылки и поделиться кодом

В среде есть функции открытия и импорта исходников, сохранения .uis, ссылки для обмена кодом и экспорта готовой HTML-страницы. При передаче кода через ссылку убедитесь, что ссылка содержит именно текущую версию программы. Длинные программы удобнее передавать отдельным файлом .uis.

14.3. Экспорт автономного HTML-приложения

Чтобы получить страницу, которая запускается без среды разработки:

1. Запустите исходник и исправьте ошибки.
2. Сохраните .uis как исходную копию.
3. Выберите «Файл» → «Скачать приложение (.html)».
4. Откройте полученный файл в браузере и проверьте основные сценарии ещё раз.
5. Перед отправкой другому человеку протестируйте страницу именно в том браузере и на том устройстве, где её будут открывать.

Экспортёр собирает отдельную HTML-страницу с исходным кодом программы и нужным встроенным runtime. Перед сохранением среда проверяет разбор исходника, выполняет сухой прогон без запуска таймеров и сетевых запросов, а затем проверяет, что встроенный JavaScript собирается. Это снижает риск ошибки экспорта, но не заменяет проверку работы после скачивания.

14.4. Что автономность означает — и чего не означает

Экспортированная HTML-страница не требует открывать редактор UIscript для работы её интерфейса. Однако это не делает внешние ресурсы локальными: картинки, видео и API-запросы по URL по-прежнему зависят от доступности сети, правил CORS и сервера. Если страница будет использоваться без интернета, внешние URL могут не работать.

> **Безопасность.** Автономный HTML — файл с исполняемым кодом. Отправляйте его только тогда, когда готовы раскрыть содержимое приложения. Не храните в нём пароли, секретные ключи и приватные данные.

14.5. Что делать, если HTML не скачивается

- Проверьте, что редактор не пуст.
- Нажмите «Запустить» и устраните ошибки синтаксиса или выполнения.
- Повторите экспорт и прочитайте сообщение в консоли.
- Если браузер блокирует загрузку, попробуйте другой браузер или используйте окно с HTML-кодом, если среда его предлагает, и сохраните содержимое в файл с расширением .html.
- Откройте сохранённую страницу как отдельный файл: проверка только в предпросмотре недостаточна.

Мини-практика

Экспортируйте один из маленьких проектов из главы 16 и проверьте три вещи: интерфейс отображается, кнопки работают, динамические надписи обновляются. Затем отключите сеть и посмотрите, что происходит с внешними изображениями.

15 Поиск ошибок и отладка

Ошибка — это не повод переписывать программу с нуля. Прочитайте текст ошибки, перейдите к указанной строке и проверьте её вместе с ближайшими строками. Причина часто находится в неправильном отступе, имени или значении, которое пришло из поля ввода.

15.1. Три категории проблем

- **Ошибка разбора** — язык не может прочитать текст программы; запуск не начинается.
- **Ошибка выполнения** — синтаксис принят, но при выполнении действия возникает проблема.
- **Предупреждение** — программа может продолжить работу, но поведение, скорее всего, не соответствует намерению.

В приложенной среде сообщение обычно содержит номер строки. Щелчок по строке ошибки в консоли или по уведомлению об ошибке помогает перейти к месту, где она появилась.

15.2. Частые ошибки разбора

Сообщение / симптом	Почему возникает	Как исправить
«Неожиданный отступ»	Команда имеет лишний отступ или строка не находится внутри подходящего блока	Выровняйте соседние команды; дочерний отступ используйте только внутри элемента или блока
«иначе не может быть без если»	`иначе` не соответствует предыдущему `если` по уровню отступа либо условие отсутствует	Проверьте вертикальное положение `если`, `иначе если`, `иначе`
«Непонятная конструкция»	Опечатка в ключевом слове или неподдерживаемая команда	Сверьте слово с кратким справочником
«Незакрытая строка»	Отсутствует закрывающая кавычка	Закройте текст кавычкой того же типа
«Неверный синтаксис таймера»	Неподдерживаемая единица времени или неполная конструкция	Используйте `каждую секунду:`, `каждые 5 сек:` или `через 2 сек:`
«Цвет нельзя использовать с если»	У свойства `цвет` указано условие	Уберите условие или примените его к `невидимый`, `жирный`, `курсив`, `центр` либо `выключен`

Пример неверной структуры:

```
окно "Пример"
    если истина
        текст "Внутри условия"
  иначе
        текст "Отступ отличается"
```

Исправленный вариант:

```
окно "Пример"
    если истина
        текст "Внутри условия"
    иначе
        текст "Отступ отличается"
```

15.3. Частые ошибки выполнения

Сообщение / симптом	Типичная причина	Что проверить
«Количество повторений должно быть числом»	Поле пусто или содержит нечисловой текст	Перед `повторить` проверить `число_ли()`
«Слишком много повторений»	Цикл запрошен больше чем на 10 000 проходов	Ограничить ввод пользователя
«Время таймера должно быть положительным числом»	Время равно нулю, отрицательное или не число	Проверить ввод и формулу интервала
«Неизвестная функция»	Опечатка, отсутствует функция или имя не совпадает	Проверить объявление и скобки
«вернуть можно использовать только внутри функции»	`вернуть` стоит вне функции	Переместить команду внутрь `функция`
«Вариант можно писать только внутри выбор»	`вариант` стоит на неправильном уровне отступа	Вложить его внутрь `выбор`
«Список не найден»	Команда `добавить` / `удалить` ссылается на другой идентификатор	Сверить имя списка посимвольно
«Разрешены только http(s)-адреса»	В запросе используется неподдерживаемая схема URL	Использовать `https://` или `http://`
Ошибка слишком глубокой рекурсии	Функция вызывает сама себя без своевременного завершения	Исправить базовый случай или использовать цикл

15.4. Предупреждения, которые нельзя игнорировать

- **Имя не определено.** Проверьте опечатку или забытое объявление через `пусть`.
- **Переменная перекрывает элемент.** Не называйте переменную так же, как поле, текст или кнопку: обращение по этому имени становится неоднозначным для пользователя и может лишить доступа к элементу.
- **Повторяющееся имя элемента.** Дайте разным элементам уникальные идентификаторы, если собираетесь к ним обращаться.
- **Свойство не поддерживается элементом.** Перенесите свойство на элемент подходящего вида.
- **Свойство стоит вне элемента.** Добавьте правильный отступ под целевой элемент.

15.5. Логические ошибки без сообщений

Такие ошибки труднее заметить: программа выполняется, но результат неправильный.

Строки складываются вместо чисел

Если поле `а` и поле `б` — поля ввода, то `а + б` может склеить текст. Исправление: `число(а) + число(б)` после проверки `число_ли`.

Значение не обновляется

Если текст содержал `{очки}`, но команда `установить текст_очков "…"` заменила содержимое, реактивная формула отключена. Удалите ручную установку текста и обновляйте переменную `очки`.

Список постоянно растёт

Если перед генерацией нового результата не очищать старый список, повторный запуск добавляет дубликаты. Перед циклом выполните `очистить имя_списка`.

Неверный индекс

Список использует индексацию с нуля: 0 — первый элемент, 1 — второй, -1 — последний при удалении. Не путайте `i` (с единицы) и `индекс` (с нуля).

15.6. Универсальный алгоритм отладки

1. **Повторите ошибку.** Сохраните копию кода и запустите ещё раз.
2. **Уточните место.** Начните со строки, которую показывает консоль, затем прочитайте строки выше и ниже.
3. **Проверьте структуру.** Отступ, двоеточие у блока, кавычки, скобки, имя.
4. **Проверьте входные данные.** Пустая строка, пробелы, нечисловой символ, невыбранный вариант.
5. **Сузьте пример.** Временно уберите лишние элементы, пока минимальный фрагмент не начнёт работать.
6. **Добавьте показать.** Выведите промежуточные значения: `показать "x={x}"`.
7. **Возвращайте части по одной.** Когда маленький пример работает, добавляйте обратно по одному блоку.

Не исправляйте одновременно пять разных проблем: иначе будет трудно понять, какое изменение помогло.

16 Практикум: четыре законченных проекта

Каждый проект собирается из команд, описанных выше. Сначала вставьте код целиком, затем запустите и выполните шаги проверки. Все строки можно редактировать: цель не только получить красивый интерфейс, но и понять, какие конструкции за что отвечают.

Проект 1. Калькулятор четырёх операций

Что изучаем: поля ввода, выпадающий выбор, проверку, ветвление и округление.

```
окно "Калькулятор"
    заголовок "Четыре операции"
    строка
        поле первое "Число A"
        выбор операция "Операция"
            вариант "+"
            вариант "-"
            вариант "*"
            вариант "/"
        поле второе "Число B"
    текст итог "Введите числа и нажмите «Посчитать»."
    кнопка "Посчитать"
        нажми:
            если не число_ли(первое) или не число_ли(второе)
                установить итог "Ошибка: введите оба числа."
            иначе
                пусть a = число(первое)
                пусть b = число(второе)
                если операция == ""
                    установить итог "Выберите операцию."
                иначе если операция == "+"
                    установить итог "Результат: {округлить(a + b, 6)}"
                иначе если операция == "-"
                    установить итог "Результат: {округлить(a - b, 6)}"
                иначе если операция == "*"
                    установить итог "Результат: {округлить(a * b, 6)}"
                иначе если b == 0
                    установить итог "На ноль делить нельзя."
                иначе
                    установить итог "Результат: {округлить(a / b, 6)}"
```

Проверьте: 12 + 3 даёт 15; 12 / 4 даёт 3; пустое поле выдаёт понятную ошибку; попытка деления на ноль не производит расчёт.

Что улучшить самостоятельно: добавить кнопку сброса полей; показать, какую операцию выбрал пользователь; ограничить количество знаков после запятой.

Проект 2. Список покупок

Что изучаем: добавление, удаление, очистку, фокус, валидацию и реактивное количество элементов.

```
окно "Покупки"
    заголовок "Список покупок"
    строка
        поле товар "Что нужно купить?"
        кнопка "Добавить"
            нажми:
                если пусто(обрезать(товар))
                    сообщение "Введите название товара."
                    фокус товар
                иначе
                    добавить покупки "{обрезать(товар)}"
                    очистить товар
                    фокус товар
    текст "Количество позиций: {длина(покупки)}"
    список покупки
    строка
        кнопка "Удалить первый"
            нажми:
                удалить покупки 0
        кнопка "Удалить последний"
            нажми:
                удалить покупки -1
        кнопка "Очистить список"
            нажми:
                очистить покупки
```

Проверьте: пустая строка не добавляется; после добавления поле очищается; счётчик количества меняется; команды удаления и очистки работают.

Почему используется `длина(покупки)`: именованный список при чтении возвращается как массив элементов. Функция `длина()` умеет возвращать число элементов массива.

Что улучшить самостоятельно: добавьте отдельную кнопку «Удалить всё с подтверждением» или запретите удаление, когда список пуст. Подумайте, какую проверку необходимо добавить, чтобы не выводить сообщение об ошибке пользователю без причины.

Проект 3. Мини-викторина

Что изучаем: состояние программы, условие, строковые функции, блокировку повторного ответа.

```
пусть очки = 0
пусть отвечено = ложь
окно "Викторина"
    заголовок "Один вопрос"
    текст "Какая планета известна как Красная планета?"
    поле ответ "Введите название планеты"
    текст результат "После ответа здесь появится результат."
    текст "Очки: {очки}"
    кнопка "Проверить ответ"
        нажми:
            если отвечено
                сообщение "Вы уже ответили на этот вопрос."
            иначе если пусто(обрезать(ответ))
                сообщение "Введите ответ."
                фокус ответ
            иначе
                отвечено = истина
                если нижний(обрезать(ответ)) == "марс"
                    очки += 1
                    установить результат "Верно!"
                иначе
                    установить результат "Не совсем. Правильный ответ: Марс."
                отключить ответ
```

Проверьте: ответы «Марс», «марс» и «МАРС» принимаются, потому что `обрезать()` убирает пробелы, а `нижний()` приводит текст к нижнему регистру. Повторный клик не начисляет очко второй раз. Поле после ответа блокируется.

Что улучшить самостоятельно: добавьте несколько вопросов; выводите процент правильных ответов; реализуйте кнопку перезапуска, которая сбрасывает `очки`, `отвечено` и поле ответа. При сбросе нужно снова выполнить `включить ответ`.

Проект 4. Обратный отсчёт

Что изучаем: переменные состояния, таймер, проверку ввода и реактивное отображение.

```
пусть осталось = 0
пусть идёт = ложь
окно "Обратный отсчёт"
    заголовок "{осталось}"
        центр
        размер 48
    поле длительность "Сколько секунд?"
    текст статус "Готов к запуску"
        центр
    строка
        кнопка "Старт"
            нажми:
                если идёт
                    сообщение "Отсчёт уже идёт."
                иначе если не число_ли(длительность)
                    сообщение "Введите количество секунд."
                    фокус длительность
                иначе если число(длительность) < 1
                    сообщение "Введите число больше нуля."
                иначе
                    осталось = пол(число(длительность))
                    идёт = истина
                    установить статус "Отсчёт запущен"
        кнопка "Сброс"
            нажми:
                идёт = ложь
                осталось = 0
                установить статус "Сброшено"
    каждую секунду:
        если идёт
            если осталось > 0
                осталось -= 1
            если осталось == 0
                идёт = ложь
                установить статус "Время вышло"
                сообщение "Отсчёт завершён!"
```

Проверьте: введите 5, нажмите «Старт» и наблюдайте, как число уменьшается. Сброс должен остановить изменение, хотя сам таймер продолжает просыпаться и проверять флаг `идёт`.

Как это устроено: таймер запускается один раз вместе с программой. Он не создаёт новый интервал при каждом нажатии; переменная `идёт` управляет тем, изменяется ли счётчик. Такой подход надёжнее, чем стартовать новый таймер при каждом щелчке.

Что улучшить самостоятельно: отображать формат мм:сс с помощью функции, которая добавляет ведущий ноль; добавить паузу; блокировать поле ввода на время отсчёта.

17 Проверка качества перед публикацией

Рабочая программа — это не только пример, который один раз запустился. Проверьте типичные сценарии, крайние значения и способ передачи проекта.

17.1. Контрольный список

Перед экспортом ответьте «да» на каждый подходящий пункт:

- Код запускается без ошибок разбора.
- Отступы показывают правильную вложенность; каждая кнопка имеет нужное событие.
- Имена полей, элементов, переменных и функций написаны одинаково везде.
- Пустой ввод обрабатывается явно.
- Числовые поля проверяются через `число_ли()` до преобразования и вычислений.
- Деление на ноль и пустой выбор дают понятное сообщение.
- Списки очищаются перед повторной генерацией, если старый результат не нужно сохранять.
- Реактивные надписи не перезаписываются `установить` без необходимости.
- Условное свойство используется только там, где оно поддерживается.
- Таймеры не создаются повторно при каждом действии пользователя.
- Сетевой запрос проверен на реальном URL, и понятны возможные проблемы CORS.
- Приложение проверено после экспорта отдельным HTML-файлом.
- Исходный файл .uis сохранён в качестве редактируемой копии.

17.2. Минимальный набор тестов для формы

Сценарий	Ожидаемое поведение
Все поля пустые	Понятная подсказка; расчёт не выполняется
В поле числа введены буквы	Сообщение об ошибке, а не бессмысленное число
Введено 0	Обрабатывается согласно смыслу поля; не смешивается с пустой строкой
Введено 1,5	`число()` преобразует десятичную запятую
Не выбран вариант	Программа предлагает сделать выбор
Галочка не установлена	Проверка согласия не проходит
Нажатие повторяется	Не происходит случайного удвоения очков, списка или таймеров
После сброса нажата кнопка снова	Приложение возвращается в согласованное начальное состояние

17.3. Производительность и ясность

Для небольших приложений UIscript не требует сложной оптимизации, но хорошие привычки всё равно важны:

- Не запускайте множество одинаковых таймеров. Используйте один периодический таймер и

переменную состояния.

- Не делайте чрезмерно длинные циклы: в среде есть ограничение числа повторов и защита от слишком большого числа операций.
- Проверяйте ввод до дорогих вычислений и сетевых запросов.
- Не повторяйте большой блок кода несколько раз, если его можно вынести в функцию.
- Делайте сообщения конкретными: «Введите второе число» лучше, чем «Ошибка».
- Используйте имена вроде `сумма`, `длительность`, `результат`, а не `x1`, `x2`, `tmp`, если имена должны быть понятны через неделю.

17.4. Границы среды разработки

Для данной версии парсера важно помнить несколько конкретных ограничений:

- Нет циклов `while`, команд `break` и `continue`.
- Цикл `повторить` ограничен 10 000 проходами.
- Глубина вложенных вызовов функций ограничена 100.
- Одновременно допускается до 50 активных таймеров.
- За один проход предусмотрена защита от чрезмерного числа операций.
- Для сетевых запросов действует тайм-аут и лимит параллельных запросов.
- Условные свойства ограничены `выключен`, `невидимый`, `жирный`, `курсив`, `центр`.

> **Примечание.** Это ограничения именно приложенной HTML-реализации, а не универсальные правила всех возможных языков с названием UIscript.

18 Полный краткий справочник

Этот раздел предназначен для быстрого поиска синтаксиса во время работы.

18.1. Элементы

Группа	Команды
Основа	`окно` , `заголовок` , `подзаголовок` , `текст` , `разделитель`
Действия	`кнопка` , `ссылка`
Ввод	`поле` , `область` , `галочка` , `ползунок`
Списки	`выбор` , `вариант` , `список`
Медиа	`картинка` , `видео`
Контейнеры	`строка` , `столбец` , `карточка`

18.2. Свойства

Тип значения	Свойства
Текст, цвет, адрес	`фон` , `цвет` , `подсказка` , `рамка` , `адрес`
Числовые размеры	`размер` , `ширина` , `высота` , `отступ` , `скругление` , `прозрачность` , `мин` , `макс` , `значение`
Флаги	`жирный` , `курсив` , `центр` , `тень` , `невидимый` , `выключен`
Условная форма	`жирный если …` , `курсив если …` , `центр если …` , `невидимый если …` , `выключен если …`

18.3. События и действия

События: `нажми:` , `измени:` , `наведи:` , `уход:` .

Действия: `сообщение` , `показать` , `установить` , `очистить` , `добавить` , `удалить` , `скрыть` , `открыть` , `переключить` , `включить` , `отключить` , `фокус` , `запрос` , `вызвать` .

18.4. Управляющие конструкции

```
пусть имя = значение
имя = новое_значение
имя += 1

если условие
    действие
иначе если другое_условие
    действие
иначе
    действие

повторить 5 раз:
    показать "Проход {i} (индекс {индекс})"

функция квадрат(x):
    вернуть x * x

вызвать моя_функция(аргумент)

каждую секунду:
    действие

каждые 500 мс:
    действие

через 3 сек:
    действие
```

18.5. Операторы

Категория	Операторы
Арифметика	`+` , `-` , `*` , `/` , `%`
Сравнение	`==` , `=` , `!=` , `>` , `<` , `>=` , `<=`
Логика	`и` , `или` , `не` ; также `and` , `or` , `not` , `&&` , `||` , `!`
Присваивание	`=` , `+=` , `-=` , `*=` , `/=` , `%=`
Группировка	`(...)`

Используйте `==` в условиях и `=` для присваивания, чтобы не путать задачи оператора.

18.6. Встроенные функции

Функция	Назначение	Пример
`число(x)`	Преобразовать строку в число; нечисловой ввод даёт `NaN`	`число("1,5")` → 1.5
`число_ли(x)`	Проверить, является ли значение корректным конечным числом	`число_ли("abc")` → ложь
`округлить(x, n)`	Округлить до n знаков; диапазон n — 0…10	`округлить(2 / 3, 2)` → 0.67
`пол(x)`	Округлить вниз до целого	`пол(3.7)` → 3
`потолок(x)`	Округлить вверх до целого	`потолок(3.1)` → 4
`абс(x)`	Модуль числа	`абс(-5)` → 5
`мин(a, b, …)`	Наименьшее из переданных чисел	`мин(3, 8)` → 3
`макс(a, b, …)`	Наибольшее из переданных чисел	`макс(3, 8)` → 8
`случайное()`	Случайное число от 0 включительно до 1 не включительно	`случайное()`
`случайное(a, b)`	Случайное целое от a до b включительно	`случайное(1, 6)`
`длина(x)`	Количество символов или элементов массива	`длина("привет")` → 6
`верхний(s)`	Перевести текст в верхний регистр	`верхний("abc")` → ABC
`нижний(s)`	Перевести текст в нижний регистр	`нижний("ДА")` → да
`обрезать(s)`	Убрать пробелы по краям	`обрезать(" да ")` → да
`пусто(x)`	Проверить пустое значение, строку, `null`, `ложь` или пустой массив	`пусто("")` → истина
`путь(obj, "a.b[0]")`	Получить вложенное свойство объекта или массива	`путь(ответ, "user.name")`
`сейчас()`	Текущее время устройства в локальном формате	`сейчас()`
`дата()`	Текущая дата устройства в локальном формате	`дата()`
`секунд_до("ГГГГ-ММ-ДД ЧЧ:ММ")`	Секунды до указанной даты и времени; отрицательное число означает, что момент прошёл	`секунд_до("2027-01-01 00:00")`

> **Уточнение о `пусто()`.** Пробелы внутри строки не удаляются автоматически. `пусто(" ")` не заменяет `пусто(обрезать(значение))`, когда требуется считать одни пробелы отсутствием ввода.

18.7. Короткие шаблоны

Следующие примеры — фрагменты, а не самостоятельные программы. Вставляйте их в подходящий блок существующего приложения. Например, `добавить задачи ...` требует заранее созданного списка `задачи` и обычно находится внутри обработчика `нажми:`; имена полей и переменных должны совпадать с вашими.

Проверка числа:

```
если не число_ли(поле_числа)
    сообщение "Введите число."
иначе
    пусть х = число(поле_числа)
```

Короткая реактивная надпись:

```
пусть счётчик = 0
текст "Счётчик: {счётчик}"
```

Добавление в список:

```
добавить задачи "{обрезать(новая_задача)}"
```

Внешний запрос:

```
запрос "https://api.example.com/data" в ответ
```

Показ условно:

```
текст предупреждение "Проверьте данные"
    невидимый если не ошибка
```

19 Задания для самостоятельной работы и решения

Попробуйте выполнить каждое задание прежде, чем читать решение. Не обязательно использовать ровно тот же способ: важно, чтобы программа корректно работала и была понятной.

Задания

1. **Приветствие.** Сделайте поле имени и кнопку. При пустом или состоящем из пробелов имени покажите просьбу ввести имя; иначе поприветствуйте человека.
2. **Чётное или нечётное.** Пользователь вводит целое число. Покажите, делится ли оно на два без остатка. Подсказка: используйте остаток `%`.
3. **Список задач.** Добавьте поле и кнопку. Пустая задача не должна добавляться. После добавления очистите поле, а возле списка покажите число задач.
4. **Генератор кнопок.** Создайте пять кнопок циклом. Каждая должна показывать собственный номер, а не номер последней кнопки.
5. **Функция скидки.** Создайте функцию, которая получает цену и процент скидки и возвращает итоговую цену. Проверьте несколько примеров.
6. **Состояние «готово».** Сделайте галочку «Выполнено» и реактивную надпись, меняющуюся в зависимости от её значения. Не используйте ручное `установить` для этой реактивной надписи.
7. **Таймер.** Сделайте счётчик, который увеличивается раз в секунду, пока переменная `идёт` равна `истина`. Добавьте кнопку «Старт / Пауза».
8. **API.** Выполните запрос к подходящему публичному API, выведите результат, затем извлеките из JSON одно вложенное поле через `путь()`.

Возможные решения

Решение 1. Приветствие

```
окно "Приветствие"
    поле имя "Введите имя"
    кнопка "Поздороваться"
        нажми:
            если пусто(обрезать(имя))
                сообщение "Введите имя."
                фокус имя
            иначе
                сообщение "Привет, {обрезать(имя)}!"
```

Решение 2. Чётность

```
окно "Чётность"
    поле число_ввода "Введите целое число"
    кнопка "Проверить"
        нажми:
            если не число_ли(число_ввода)
                сообщение "Введите число."
            иначе если число(число_ввода) % 2 == 0
                сообщение "Число чётное."
            иначе
                сообщение "Число нечётное."
```

Решение 3. Список задач

```
окно "Задачи"
    поле новая_задача "Текст задачи"
    кнопка "Добавить"
        нажми:
            если пусто(обрезать(новая_задача))
                сообщение "Введите задачу."
            иначе
                добавить задачи "{обрезать(новая_задача)}"
                очистить новая_задача
    текст "Всего: {длина(задачи)}"
    список задачи
```

Решение 4. Генератор кнопок

```
окно "Пять кнопок"
    строка
        повторить 5 раз:
            кнопка "№ {i}"
                нажми:
                    сообщение "Выбрана кнопка {i}"
```

Решение 5. Функция скидки

```
функция цена_со_скидкой(цена, процент):
    вернуть цена * (1 - процент / 100)

окно "Скидка"
    текст "1000 со скидкой 15%: {округлить(цена_со_скидкой(1000, 15), 2)}"
```

Решение 6. Выполнено или не выполнено

```
окно "Статус"
    галочка выполнено "Задача выполнена"
    текст "Готово"
        невидимый если не выполнено
    текст "Ещё в работе"
        невидимый если выполнено
```

Решение 7. Секундомер

```
пусть секунды = 0
пусть идёт = ложь
окно "Секундомер"
    заголовок "Время: {секунды} сек"
    кнопка "Старт / Пауза"
        нажми:
            идёт = не идёт
    кнопка "Сброс"
        нажми:
            секунды = 0
            идёт = ложь
каждую секунду:
    если идёт
        секунды += 1
```

Решение 8. API

Поскольку доступность публичных API и их CORS-настройки меняются, универсального гарантированно работающего URL для всех браузеров нет. Возьмите адрес API, который разрешает запросы из браузера, затем используйте шаблон:

```
пусть ответ = null
окно "API"
    кнопка "Загрузить"
        нажми:
            запрос "https://ВАШ_API/путь" в ответ
    текст "Ответ: {ответ}"
    текст "Имя: {путь(ответ, 'user.name')}"
```

Замените адрес на реальный, а `user.name` — на путь к существующему полю в ответе. Если запрос не проходит, проверьте консоль, код HTTP, разрешение CORS и формат данных. URL в примере намеренно не является действующим сервисом.

Заключение

Вы освоили основные элементы и команды UIscript: построение интерфейса, переменные и вычисления, условия, события, циклы, функции, таймеры, реактивность, списки и запросы.

Следующий шаг — постоянно создавать небольшие законченные проекты. Лучше три простых приложения, в которых вы понимаете каждую строку, чем один огромный пример, который страшно менять.

Рабочий порядок для нового проекта:

1. Сначала опишите интерфейс без логики.
2. Дайте имена элементам, значения которых понадобятся в программе.
3. Добавьте одно событие и добейтесь его работы.
4. Подключите переменные, условия и валидацию ввода.
5. Добавьте динамический интерфейс и необходимые таймеры.
6. Проверьте крайние случаи и экспортируйте автономный HTML.
7. Сохраните исходник .uis отдельно от готового HTML.

Главная цель — не запомнить все команды, а научиться разбивать задачу на маленькие понятные шаги и проверять каждый из них.

> **Примечание о совместимости.** Книга ориентирована на поведение языка версии 1.0, реализованное в файле uis_1-0.html. Если реализация будет изменена, новые версии команд или ограничения следует проверять по соответствующему HTML-файлу среды разработки.